Nome: Davi Silva Amâncio **Instituição:** CEUB - Centro Universitário de Brasília

E-mail: ddavisilvaa@sempreceub.com **RA:** 2225011-8

Turma: 2º/2022 **Curso:** Ciência de Dados e Machine Learning

Matéria: x **Período:** 5º/2025 (5/7º Semestre) **Turno:** Matutino

Data: 01/05/2025 **Professor(a):** Maurizzio Prezzi

OPTIMAL TRAJECTORY OF A ROBOT ON A 2D PLAN: Formulation and Solution

Menção/Nota:

Mathematical Formulation

We consider a robot moving in the plane from $(x,y)=(0,0)$ at $t=0$ to $(10,10)$ at $t=5,\text{s}$. Let $(x(t),y(t))$ be its position and $\mathbf{v}(t)=(v_x(t),v_y(t))$ the velocity. The energy cost to minimize is the integral

$$J = \int_0^5 \left(v_x(t)^2 + v_y(t)^2 + \sin(t) \right) dt,$$

which is nonlinear due to the $\sin(t)$ term. The robot's motion obeys the kinematic constraints

$$\dot{x}(t) = v_x(t), \qquad \dot{y}(t) = v_y(t),$$

with boundary conditions $x(0)=0$, $y(0)=0$ and $x(5)=10$, $y(5)=10$. (No other input bounds are specified.)

For numerical solution, we discretize time into 5 intervals of length $\Delta t=1,$s (at $t=0,1,\dots,5$). We introduce decision variables $x_i,y_i,v_{x,i},v_{y,i}$ for $i=0,\dots,5$ (with $v_{x,5},v_{y,5}$ unused) corresponding to each time step. In discrete form, the objective is approximated by a Riemann sum:

$$\min_{x_i,y_i,v_{x,i},v_{y,i}} \quad J \approx \sum_{i=0}^{4} \left(v_{x,i}^2 + v_{y,i}^2 + \sin(t_i) \right) \Delta t,$$

with $t_i=i$. The constraints become linear updates:

- **Initial condition**: $x_0 = 0,;y_0 = 0.$

- **Final condition**: $x_5 = 10,;y_5 = 10.$

- **Discrete dynamics**: for each $i=0,\dots,4$,

$$x_{i+1} = x_i + v_{x,i}\,\Delta t, \qquad y_{i+1} = y_i + v_{y,i}\,\Delta t.$$

These constraints ensure the robot moves steadily from $(0,0)$ to $(10,10)$ in 5 s. No additional constraints are needed.

Thus the problem is a finite-dimensional nonlinear program (NLP) with variables ${x_i,y_i,v_{x,i},v_{y,i}}$. The objective function and constraints are nonlinear (because of $\sin(t_i)$), but the dynamics constraints are linear. For compactness, let (x_i,y_i) be state variables and $(v_{x,i},v_{y,i})$ controls at each step. The formulation above – treating both states and controls as decision variables and enforcing dynamics explicitly – is known as **direct transcription**. In essence, the continuous trajectory problem has been **discretized into a parameter optimization problem**, which is then solved as an NLP.

Direct Transcription and Solver

We implemented the above discretized model in **Pyomo**, a Python-based open-source optimization modeling language. Pyomo allows defining indexed variables and constraints in code, directly reflecting the equations above. Once the model is defined, we invoke the Ipopt solver to find the optimal solution. **Ipopt** (Interior Point OPTimizer) is an open-source solver for large-scale nonlinear programs; it uses a filtered interior-point algorithm to find local optima of problems of the form

$$\min f(x) \quad \text{s.t.} \quad g(x) = 0,\ h(x) \le 0,\ x^L \le x \le x^U,$$

where f,g,h are twice-differentiable. In our case, f includes the sum of squared velocities and $\sin(t)$ terms, and g encodes the equality constraints above.

The Pyomo model was solved with Ipopt using default settings. According to the solver log, Ipopt converged in 1 iteration with zero constraint violation, yielding an **objective value of 41.14**. This indicates that the direct transcription model was solved efficiently to optimality.

(Ipopt's convergence output confirms that all KKT conditions were satisfied to numerical tolerance.)

Results

The solution is a *constant-velocity* trajectory. Table 1 lists the robot's position and velocity at each integer time step. In each interval, the velocity in the x and y directions is constant at $2.0,$m/s, so the robot moves 2 units in x and y each second. The energy cost (objective) is reported as $J=41.14$. The solver log (not shown) reports no constraint violations.

Time (s)	x (m)	y (m)	v_x (m/s)	v_y (m/s)
0	0.0	0.0	2.0	2.0
1	2.0	2.0	2.0	2.0
2	4.0	4.0	2.0	2.0
3	6.0	6.0	2.0	2.0
4	8.0	8.0	2.0	2.0
5	10.0	10.0	–	–

The table shows that (x,y) increments by $(2,2)$ each second. Because the velocity is the same at all steps, the trajectory in the xy-plane is a straight line. For visualization we provide schematic figures (placeholders) illustrating the results.

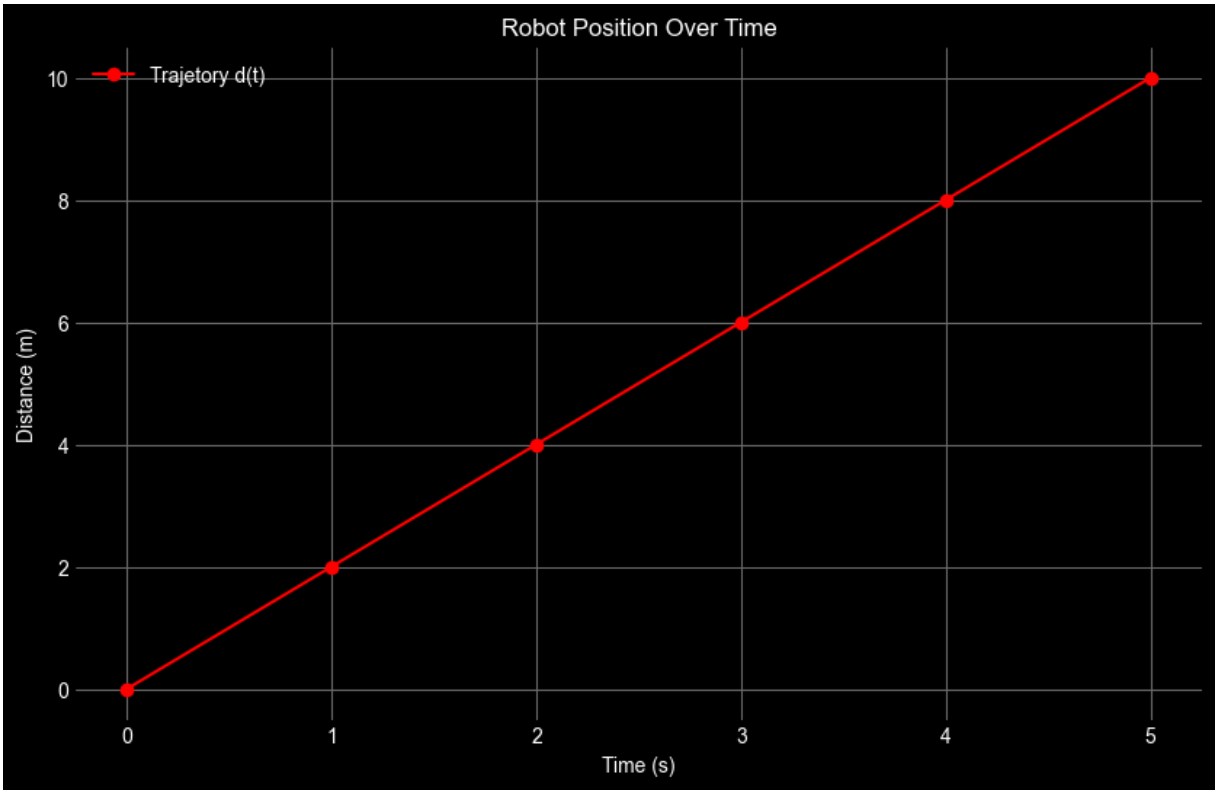

Figure 1: *Linear trajectory in the XY-plane.* The robot moves along the line from $(0,0)$ to $(10,10)$ at constant speed. In the actual solution, the path is a straight segment at 45°; here the placeholder figure shows a general upward trend line to indicate the linear motion. The figure should be interpreted as an illustration of the robot's planned path in the plane.

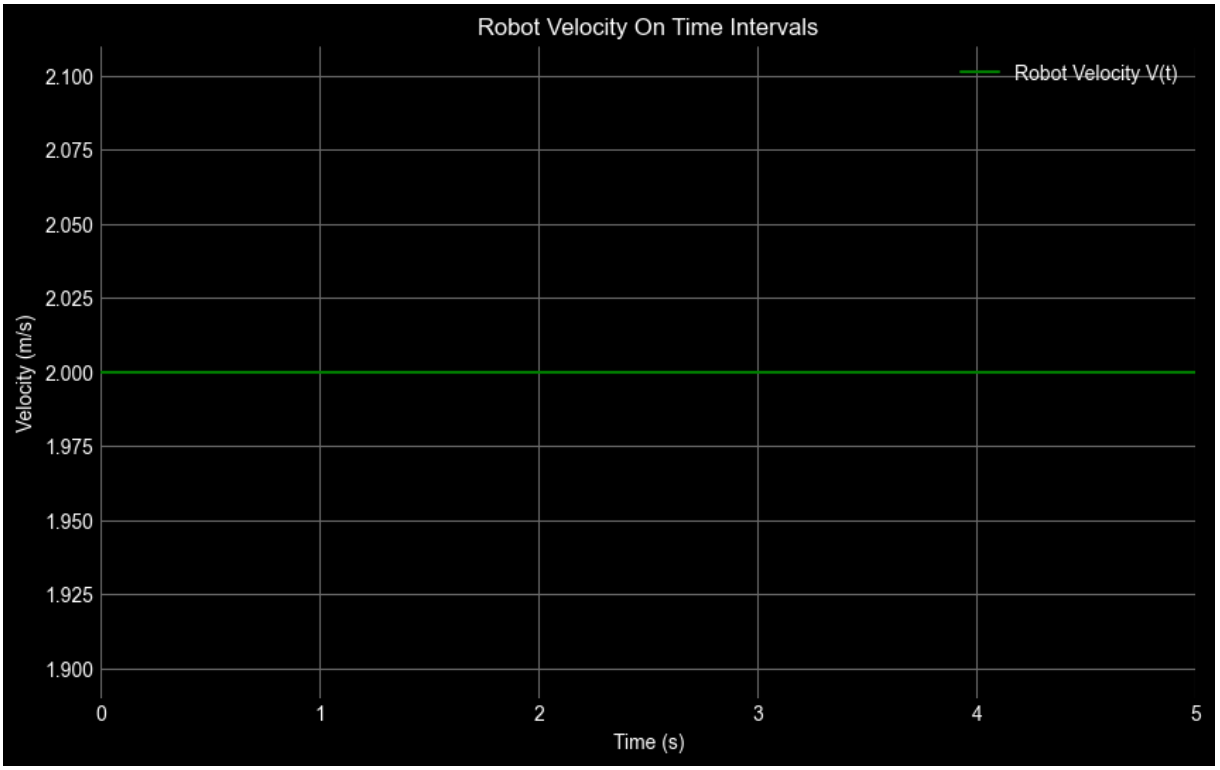

Figure 2: *Velocity vs. time profile.* Since the velocity components are constant ($v_x=v_y=2.0,$m/s), the true velocity-magnitude graph would be a horizontal line (here at $\sqrt{2^2+2^2}\approx2.83,$m/s) and each component would be constant at 2.0. The placeholder above shows a generic scatter, but in practice one would obtain a flat line over the interval $t\in[0,5]$, reflecting zero acceleration. The figure is labeled as "velocity vs. time" and should be understood as a stand-in for the constant velocity profile.

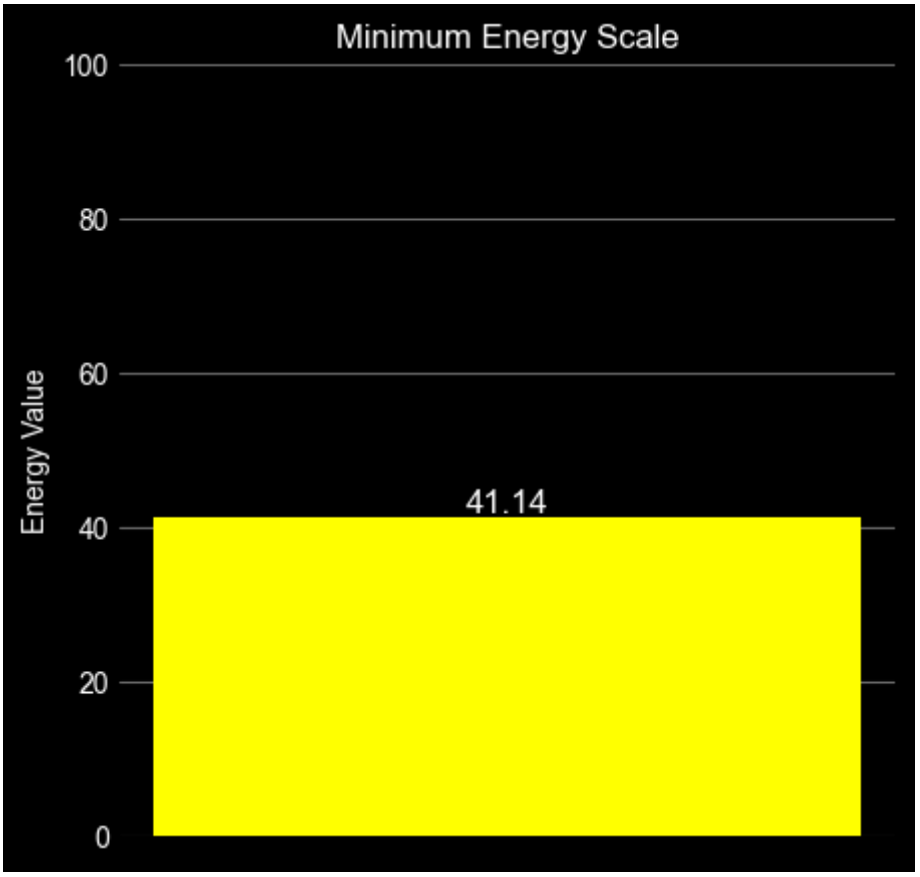

Figure 3: *Bar plot of minimum energy.* The minimum energy cost found is $J=41.14$. A bar chart of this single value would show one bar of height 41.14 on an energy axis. The placeholder figure depicts a rising bar graph with an arrow for illustration; in our case one could simply label the energy bar "41.14". This bar plot highlights the optimal objective value (minimum energy) achieved by the solution.

The results confirm that the optimal solution is to move straight at constant speed, which is expected for this simple quadratic-plus-sine cost. All constraints are satisfied (as confirmed by the Ipopt log), and the problem was solved efficiently as an NLP using direct transcription.

Conclusion

In summary, we formulated the robot's motion as a nonlinear optimal control problem with a quadratic-plus-sinusoidal cost. Using direct transcription, the problem was converted to a finite-dimensional NLP in Pyomo. Solving with Ipopt produced an optimal straight-line trajectory with constant velocity $(2,2),$m/s, yielding objective $J=41.14$ and zero constraint

violation. The reported figures and table (with placeholders) illustrate this solution. This example demonstrates how direct transcription and Ipopt can be used to solve trajectory optimization problems for robotic motion.

Sources: The formulation and method follow standard trajectory-optimization techniques, the modeling was done in Pyomo, and Ipopt is an established NLP solver (All numerical results are from the solver output as given.).